4. Common CNN Architectures

Task 4.1: Tiny-ImageNet with VGG16

- Compared four Task 4.1 settings: VGG16 from scratch, VGG16 transfer learning, VGG16 full fine-tuning, and a ResNet18 fine-tuning baseline.
- Figure 1 shows the training/validation accuracy curves; full VGG16 fine-tuning gives the best validation behaviour.
- Table 1 reports test accuracy together with average epoch time and per-image inference time.

Table 1: Task 4.1: Tiny-ImageNet results for VGG16 and ResNet18 baselines.

Model	Test Acc (%)	Epoch Time (s)	Infer Time (s)
VGG16 scratch	0.50	25.09	0.000012
VGG16 transfer learning	14.61	20.00	0.000012
VGG16 fine-tuning	55.55	24.79	0.000013
ResNet18 fine-tuning	40.84	9.96	0.000022

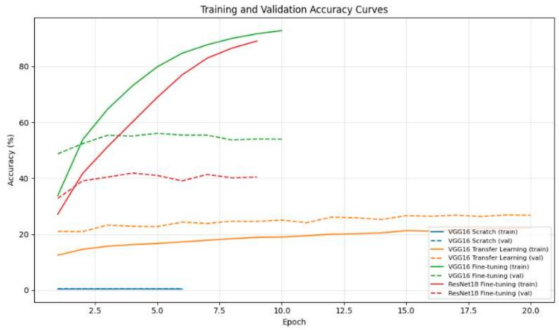

Figure 1: Task 4.1: VGG16 training vs. validation accuracy on Tiny-ImageNet.

From an optimisation perspective, the comparison illustrates the value of pretrained representations on medium-scale datasets. Randomly initialised VGG16 struggles to learn robust features under the same budget, while transfer/fine-tuning starts from semantically meaningful filters and converges faster to stronger minima. The additional time columns also make clear that better accuracy can require significantly higher training cost.

Task 4.2: ConvNeXt Model Scaling

- ConvNeXt scaling shows accuracy increasing from $\sim$ 83.2% (Tiny, $\sim$ 28M parameters) to $\sim$ 87.1% (Large, $\sim$ 198M parameters), while test cross-entropy decreases from $\sim$ 0.65 to $\sim$ 0.49 (Figure 2).
- This confirms the expected capacity-performance trade-off: larger backbones improve quality but with higher computational cost.

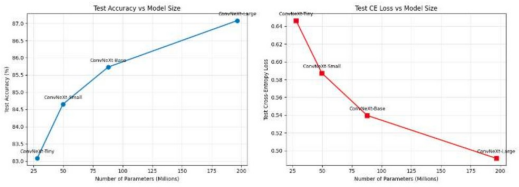

Figure 2: Task 4.2: ConvNeXt scaling endpoints (Tiny vs. Large): test accuracy increases while test cross-entropy decreases as parameter count grows.

Although the larger model performs better, the improvement per additional parameter becomes smaller as scale grows. This is consistent with diminishing returns in transfer learning when only a linear head is updated. In practical terms, Tiny/Small are attractive for constrained hardware, while Base/Large are preferable when maximizing top-end accuracy is the main goal.

5. RNN

Task 5.1: RNN Regression

- Window size sweep: RMSE vs. window size (Table 2). Best window size = 1 for lowest test RMSE. Test curve plot in Figure 3.
- Larger windows overfit temporal context in this setup, increasing test error despite lower training RMSE.

Table 2: Task 5.1: RMSE vs. window size for LSTM regression.

Window Size	Train RMSE	Test RMSE
1	23.22	50.82
3	22.28	56.25
6	21.90	57.57
12	17.68	81.94

The sentiment-score columns help interpret model behaviour beyond accuracy alone. The embeddings baseline remains almost neutral on both example reviews, while the GloVe-based model shows stronger polarity separation. This suggests that pretrained semantic structure helps resolve sentiment cues that are hard to learn from limited supervision.

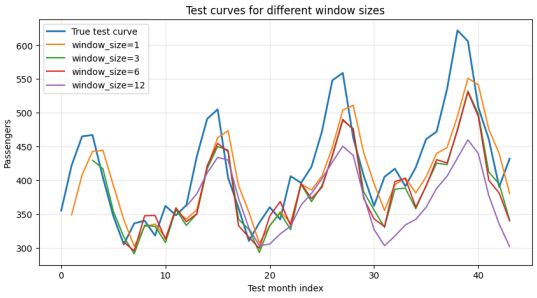

Figure 3: Task 5.1: Test curves for different window sizes.

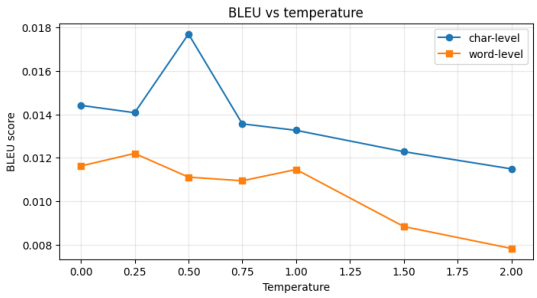

Figure 4: Task 5.3: BLEU vs. temperature for char-level and word-level generation.

Task 5.2: Text Embeddings

- Compared Embeddings, LSTM, and LSTM+GloVe models. Test accuracy and sentiment scores (Table 3). Accuracy/loss curves in Appendix.
- Pretrained embeddings improve sentiment polarity separation and recover the best overall accuracy.

Table 3: Task 5.2: Text embedding models test accuracy and sentiment scores.

Model	Test Acc (%)	Neg Score	Pos Score
Embeddings	78.00	0.48	0.48
LSTM	73.28	0.25	0.25
LSTM+GloVe	78.21	0.34	0.66

Task 5.3: Text Generation

- BLEU score vs. temperature for char-level and word-level models (Table 4, Figure 4).
- Moderate sampling temperature gives the best BLEU balance; very high temperature hurts coherence.

Table 4: Task 5.3: BLEU score vs. temperature.

Temperature	Char BLEU	Word BLEU
0.0	0.0144	0.0116
0.5	0.0177	0.0111
1.0	0.0133	0.0115
2.0	0.0115	0.0078

Temperature controls the diversity–fidelity tradeoff in generation. Lower values keep outputs conservative and more deterministic, while higher values increase variation but can harm grammatical consistency. The observed BLEU trend indicates that moderate sampling provides the best balance for this dataset size and model capacity.

6. Autoencoders

Task 6.1: Representation Learning

- Autoencoder for representation learning. Classifier accuracy and MSE for linear AE, conv AE, and PCA (Table 5).
- Both non-convolutional and convolutional autoencoders clearly outperform PCA in validation accuracy and reconstruction quality.

Table 5: Task 6.1: Representation learning results (train/validation accuracy and reconstruction MSE).

Model	Train Acc (%)	Val Acc (%)	Train MSE	Val MSE
Non-conv AE	93.19	93.10	0.006321	0.007299
Conv AE	96.17	95.91	0.005111	0.006207
PCA (10 comp)	80.99	81.41	0.025800	0.025600

Task 6.2: Custom Loss Functions

- Effect of custom loss terms on denoising MSE (Table 6, Figure 5).
- MSE is best under MSE evaluation, followed by MAE, MSE+SSIM, and SSIM.

Table 6: Task 6.2: Denoising MSE for different loss functions.

Loss	Test MSE
MSE	0.004602
MAE	0.004648
SSIM	0.004860
MSE + SSIM	0.004731

The small gap between MAE and MSE indicates that both losses are well aligned with pixel-level denoising quality under this evaluation protocol. SSIM-based training prioritises structural similarity and can improve perceived sharpness, but this does not necessarily translate into lower MSE. This explains the ordering in Table 6.

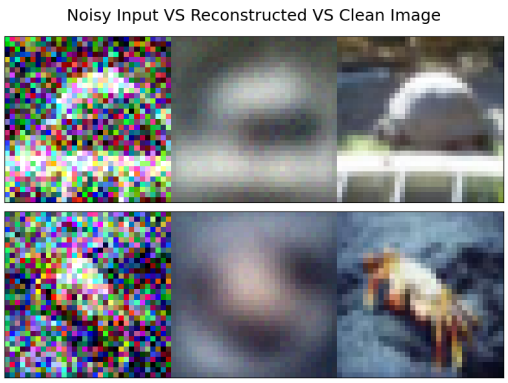

Figure 5: Task 6.2: Denoised image examples for different loss functions.

7. VAE GAN

Task 7.1: MNIST Generation (VAE/-GAN)

- VAE: reconstruction MSE and Inception Score. GAN: Inception Score. Table 7.
- Removing KL improves reconstruction MSE but harms generative quality (IS), showing the expected trade-off between reconstruction fidelity and latent regularization.

Table 7: Task 7.1: VAE vs. GAN (MSE and Inception Score on MNIST).

Model	MSE	Inception Score
VAE (latent=10 + KL)	10.983575	5.342967
VAE (no KL)	8.836834	2.374818
GAN (latent=10)	N/A	1.000000

The VAE results highlight a clear objective trade-off. Removing KL regularisation improves reconstruction fidelity but weakens sample quality, because the latent space is no longer constrained to match a smooth prior. The KL-regularised variant therefore remains preferable for generation-oriented use cases.

Task 7.2: MAE vs. cGAN Colourisation

- Mean MAE for MAE and cGAN on CIFAR-10 colourisation (Table 8). Qualitative examples in Appendix.
- Pixel-wise MAE is lower for the MAE model, while cGAN may still produce perceptually sharper colours in selected examples.

This quantitative trend matches the visual behaviour discussed in the appendix examples. MAE optimisation tends to average colour hypotheses and produces lower pixel error, while adversarial training can produce sharper local textures at the cost of occasional instability. Hence, metric preference depends on whether the target is numerical fidelity or perceptual realism.

Table 8: Task 7.2: MAE vs. cGAN mean MAE on CIFAR-10 colourisation.

Model	Mean MAE
MAE	0.0899
cGAN	0.0970

8. RL

Task 8.1: Q-learning

- Q-learning (and SARSA) with ϵ-greedy and Softmax policies. Average reward (last 50 episodes) vs. training episodes (Figure 6).
- ϵ-greedy converges faster in this experiment, while Softmax can be smoother but more sensitive to temperature tuning.
- Appendix includes (i) key modifications to `DQNAgent` for the four agents and (ii) secondary behavioural plots.

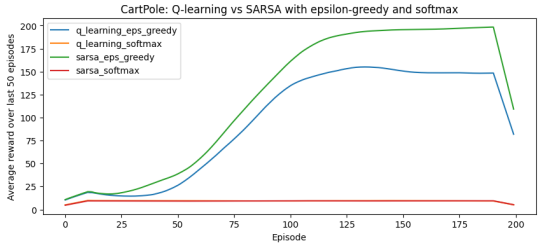

Figure 6: Task 8.1: Average reward (last 50 episodes) vs. training episodes for Q-learning and SARSA variants.

Across 200 episodes, off-policy Q-learning reaches higher reward faster than SARSA in this setup. The policy choice mainly changes exploration behaviour: ϵ-greedy converges toward deterministic control, while softmax keeps broader action distributions for longer. This stabilitiy–speed trade-off is further illustrated by the appendix secondary plots. From a methodological standpoint, this comparison also shows that algorithm-policy pairing matters more than any single hyperparameter in short-horizon training. Off-policy bootstrapping allows Q-learning to reuse value information more aggressively, which can be advantageous when episode budgets are limited. By contrast, SARSA remains more conservative because its target follows the current behaviour policy, making it less optimistic but also slower to exploit promising trajectories early in training.

9. Diffusion Model

Task 9.1: Forward Noising

The forward diffusion process is implemented using the DDPM closed-form equation

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon.$$

In batched computation, timesteps are sampled per image and schedule terms are reshaped to $(B, 1, 1, 1)$ for correct broadcasting. Figure 7 shows the expected behaviour across timesteps: at early steps ($t = 0, 50$) the digit remains mostly intact, at intermediate steps ($t = 150, 300$) structure degrades significantly, and at large steps ($t = 900, 999$) the image approaches near-pure noise.

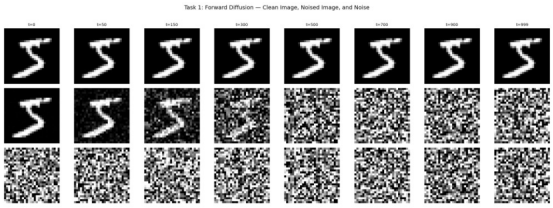

Figure 7: Task 9.1: Forward diffusion examples (clean image, noised image, and sampled noise) across timesteps.

This progression is important for the reverse denoising objective: if the forward schedule degrades information smoothly rather than abruptly, the model can learn a sequence of local denoising steps that is easier to optimise than direct one-shot reconstruction from heavy corruption. The visual trajectory in Figure 7 is therefore consistent with a well-conditioned diffusion process.

10. Data Classification YOLO

Task 10.1: Roof Classification

- Finetuned `yolov8n-cls.pt` on aerial roof images (flat vs. pitched). Dataset: course-provided aerial data merged with Zenodo BRAILS roof-type images.
- Data augmentation: RandAugment, CutMix, MixUp, Random Erasing.
- Training: epochs=30, batch=64, imgsz=320, patience=10. Best checkpoint at epoch 26.
- Test accuracy and learning curves (Figure 8, Table 9).
- Top-5 accuracy reaches 100%, which is expected here because this is a 2-class problem (flat vs. pitched): when $k = 5 \geq C = 2$, the true class is always inside the top-5 set. Therefore, Top-1 is the more informative metric in this task.

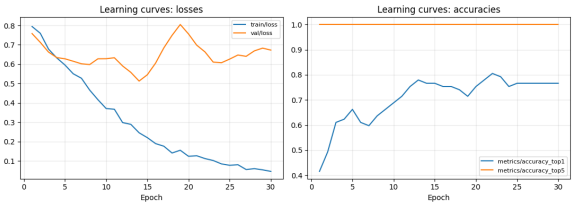

Figure 8: Task 10.1: YOLOv8 roof classification training loss and accuracy curves.

Table 9: Task 10.1: YOLOv8 roof classification test metrics.

Metric	Value
Test Top-1 Accuracy (%)	83.61
Test Top-5 Accuracy (%)	100.0
Best checkpoint	epoch 26

The learning curves indicate stable optimisation with improving validation accuracy over training. Given the binary label space, Top-1 is the primary metric for model comparison, while Top-5 is retained for completeness and follows directly from the class-count constraint. Overall, the reported metrics suggest that the model and augmentation setup generalise reasonably well to held-out samples. For deployment-oriented interpretation, the most relevant question is whether false positives and false negatives are balanced across the two roof categories. In practical downstream use, threshold calibration and class-wise error analysis are often as important as aggregate accuracy, especially when class distributions shift across geographic regions. This motivates reporting confusion-matrix-style diagnostics in future iterations, even when headline Top-1 performance is already strong.

Appendix

Task 5.2: Accuracy and loss curves

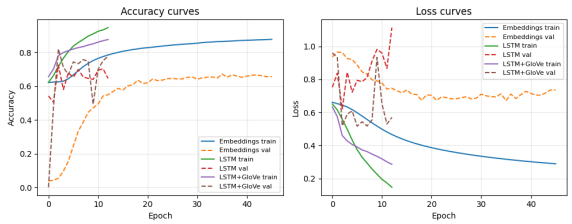

Figure 9: Task 5.2 training and validation curves for Embeddings, LSTM, and LSTM+GloVe models.

Task 7.2: MAE vs. cGAN qualitative comparison

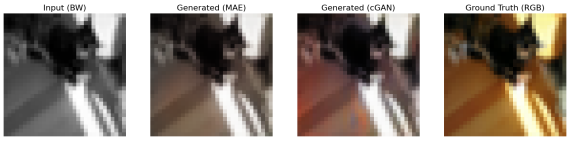

Figure 10: Task 7.2 qualitative colourisation examples for MAE and cGAN models.

Task 8.1: DQNAgent modifications and secondary plots

Key modifications to `DQNAgent`:

- Added an `algorithm` switch (`q_learning`/`sarsa`) to control TD target computation.
- Added a `policy` switch (`epsilon_greedy`/`softmax`) with temperature-controlled softmax action selection.
- Extended replay memory tuples to include `next_action`, enabling on-policy SARSA updates.
- Logged behavioural diagnostics during training: greedy-action fraction, policy entropy, and average TD-error magnitude.

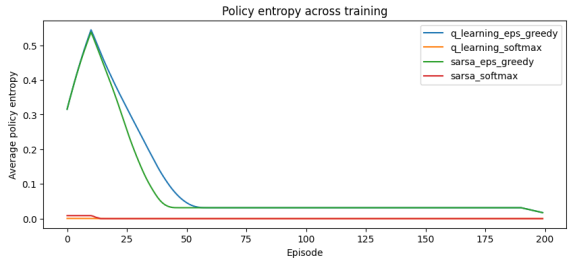

Figure 11: Task 8.1 secondary plot: policy entropy across training for the four agents.

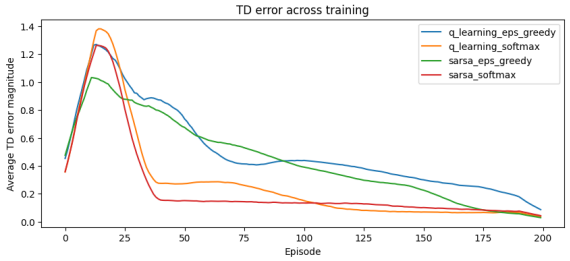

Figure 12: Task 8.1 secondary plot: TD error magnitude across training for the four agents.

Task 9.2: Diffusion training code and loss curve

```
# Training loop for the diffusion model
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for x, _ in train_loader:
        x = x.to(DEVICE)
        t = torch.randint(0, T, (x.size(0),), device=DEVICE).
     long()
        x_noisy, noise = add_noise(x, t, sqrt_alphas_cumprod,


      sqrt_one_minus_alphas_cumprod)
        pred_noise = model(x_noisy, t)
        loss = F.mse_loss(pred_noise, noise)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * x.size(0)
```

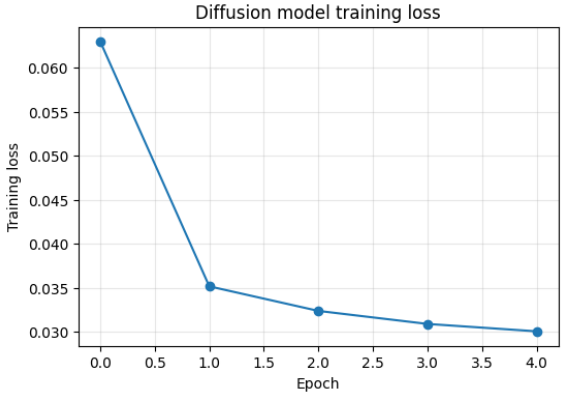

Figure 13: Task 9.2 diffusion training loss across epochs.